Route Guards

protecting pages, and finishing the auth story

DR
Dinesh Rawat
Instructor · Shorter Loop × Snapied

Yesterday we built login: the user signs in, gets a **token**, and the interceptor sends it with every request. But there is still a hole. Right now anyone can type `/profile` in the address bar and open it, even without logging in. A **route guard** closes that hole. It is the last piece of the auth story: login gets the token, the interceptor uses it, and the guard protects the pages.

1 What is a route guard?

A **route guard** is a small piece of logic that runs **before** a route opens, and decides whether to allow it. If the check passes, the page loads. If not, the guard blocks it and usually sends the user somewhere else, like the login page.

In one line: a guard is a security gate in front of a route. It answers one question, "is this user allowed in?", before the page is shown.

The ResumeForge problem it solves: the profile page should only open for a logged-in user. Without a guard, someone can skip login by typing the URL directly. The guard checks for a token first, and turns them away if it is missing.

2 Making a guard (Angular 13 style)

A guard is a service that implements `CanActivate`. The CLI creates one: `ng generate guard auth` (choose `CanActivate` when asked). Here is the guard, using the `AuthService` from yesterday.

AUTH.GUARD.TS

```
import { Injectable } from "@angular/core";
import { CanActivate, Router } from "@angular/router";
import { AuthService } from "./auth.service";

@Injectable({ providedIn: "root" })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (this.auth.getToken()) {
      return true;          // token exists, allow the page
    }

    // no token, send them to login and block the page
    this.router.navigate(["/login"]);
    return false;
  }
}
```

The `canActivate` method returns **true** to allow the route or **false** to block it. Here we check for a token: if it is there, allow; if not, redirect to login and return false. Simple gate.

3 Putting the guard on a route

Add `canActivate` to the route you want to protect, in the routing module:

APP-ROUTING.MODULE.TS

```
import { AuthGuard } from "./auth.guard";

const routes: Routes = [
  { path: "login", component: LoginComponent },
  {
    path: "profile",
    component: ProfileComponent,
    canActivate: [AuthGuard],     // protected
  },
  { path: "", redirectTo: "login", pathMatch: "full" },
];
```

Now, before `/profile` opens, Angular runs `AuthGuard`. Logged in? The page shows. Not logged in? Straight to login. You can put the same guard on any number of routes.

4 The reverse guard: NoAuthGuard

There is a second, opposite need. If a user is **already logged in**, they should not see the login or signup page again. Typing `/login` when you are already in should send you to your profile, not show the form. A **NoAuthGuard** does exactly this: it allows the page only when there is **no** token.

NO-AUTH.GUARD.TS

```
import { Injectable } from "@angular/core";
import { CanActivate, Router } from "@angular/router";
import { AuthService } from "./auth.service";

@Injectable({ providedIn: "root" })
export class NoAuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (!this.auth.getToken()) {
      return true;          // no token, allow login/signup page
    }

    // already logged in, send them to profile
    this.router.navigate(["/profile"]);
    return false;
  }
}
```

Notice it is the mirror image of AuthGuard. AuthGuard allows the page **when a token exists**. NoAuthGuard allows the page **when a token does not exist**. One check, flipped.

Put it on the login and signup routes:

APP-ROUTING.MODULE.TS

```
const routes: Routes = [
  { path: "login",  component: LoginComponent,  canActivate: [NoAuthGuard] },
  { path: "signup", component: SignupComponent, canActivate: [NoAuthGuard] },
  { path: "profile", component: ProfileComponent, canActivate: [AuthGuard] },
  { path: "", redirectTo: "login", pathMatch: "full" },
];
```

The two guards side by side:

AuthGuard protects private pages (like profile). No token? Go to login.
NoAuthGuard protects the login and signup pages. Already have a token? Go to profile.

Together they make navigation feel right: logged-out users cannot reach private pages, and logged-in users are not shown the login form again.

5 The full auth story, together

1. Login checks the user and gives a token, which we save.

2. The interceptor attaches that token to every request, so the server knows who is asking.

3. The guard protects the pages, so only a logged-in user can open them.

Three pieces, one system. Yesterday we did the first two. The guard completes it.

6 The main types of guard

`CanActivate` is the one you will use most, but it helps to know the family:

Guard	Runs when	Used for
`CanActivate`	before entering a route	block a page if not logged in
`CanDeactivate`	before leaving a route	warn about unsaved changes
`CanActivateChild`	before entering child routes	protect a group of pages at once
`Resolve`	before a route loads	fetch data so the page opens ready

A nice second one: `CanDeactivate` can ask "You have unsaved changes, leave anyway?" when a user tries to navigate away from a half-filled form. Useful in a resume builder like ResumeForge.

7 Remember this

A route guard is a gate that runs before a route and decides if the user may enter.

CanActivate is the common one: return `true` to allow, `false` to block.

The auth guard checks for a token; if missing, it redirects to login and blocks the page.

Attach it with `canActivate: [AuthGuard]` on the route.

NoAuthGuard is the reverse: it keeps logged-in users away from the login and signup pages, sending them to profile instead.

The whole story: login gets the token, the interceptor sends it, the guard protects the pages.

8 Homework

Create the AuthGuard with `ng g guard auth` and add the token check.

Protect the profile route with `canActivate: [AuthGuard]`. Log out, try opening `/profile` by URL, and confirm it sends you to login.

Log in, then try again, and confirm the profile page now opens.

Add a NoAuthGuard and put it on the login and signup routes. Log in, then try opening `/login` by URL, and confirm it sends you to profile.

Bonus: read about `CanDeactivate` and think how you would warn a user leaving a half-filled resume form in ResumeForge.